

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 5
“REKURSIF”



DISUSUN OLEH:
NADIFA AZKHIA SYARIF
109082530002
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Rekursi merupakan salah satu teknik dalam pemrograman di mana sebuah fungsi atau prosedur dapat memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Teknik ini bekerja dengan cara memecah masalah menjadi bagian yang lebih kecil hingga mencapai kondisi tertentu yang disebut sebagai *base-case*, yaitu kondisi penghentian dari proses rekursif. Selain itu, terdapat *recursive-case* yang merupakan bagian di mana fungsi melakukan pemanggilan terhadap dirinya sendiri (Munir, 2011).

Menurut Cormen et al. (2009), rekursi banyak digunakan dalam perancangan algoritma karena mampu menyederhanakan penyelesaian masalah yang memiliki pola berulang atau bersifat divide and conquer. Namun, penggunaan rekursi perlu memperhatikan efisiensi karena dapat menyebabkan pemanggilan fungsi yang berulang dan meningkatkan kompleksitas waktu.

Salah satu contoh penerapan rekursi adalah pada deret Fibonacci. Deret Fibonacci merupakan barisan bilangan yang setiap suku ke- n diperoleh dari penjumlahan dua suku sebelumnya, dengan nilai awal 0 dan 1. Secara matematis, deret ini dapat dirumuskan sebagai $S_n = S_{n-1} + S_{n-2}$. Pola ini menunjukkan adanya ketergantungan terhadap hasil sebelumnya, sehingga sangat sesuai diimplementasikan menggunakan pendekatan rekursif (Sedgewick & Wayne, 2011).

Dalam implementasinya, fungsi rekursif Fibonacci memiliki dua kondisi dasar, yaitu ketika n bernilai 0 dan 1. Untuk nilai lainnya, fungsi akan memanggil dirinya sendiri dengan parameter yang lebih kecil, yaitu $n-1$ dan $n-2$, kemudian menjumlahkan hasilnya. Meskipun konsep ini sederhana, metode rekursif memiliki kelemahan dalam hal efisiensi karena adanya perhitungan berulang (Kernighan & Ritchie, 1988).

Selain digunakan pada deret Fibonacci, rekursi juga banyak diterapkan dalam berbagai kasus lain seperti perhitungan faktorial, traversal struktur data seperti tree, serta pembentukan pola. Oleh karena itu, pemahaman konsep rekursi menjadi sangat penting dalam pengembangan algoritma dan pemrograman (Modul Praktikum Algoritma dan Pemrograman 2).

CONTOH SOAL

1. Latihan1

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int){
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Output:

```
5
4
3
2
1
```

Deskripsi Program:

Program ini bertujuan untuk menampilkan barisan bilangan dari suatu nilai input *n* hingga 1 menggunakan metode rekursif. Program dimulai dengan membaca nilai *n* dari pengguna, kemudian memanggil sebuah fungsi rekursif yang menerima parameter berupa bilangan tersebut. Di dalam fungsi, terdapat kondisi dasar (base-case) yaitu ketika bilangan bernilai 1, maka program akan mencetak angka 1 dan menghentikan proses rekursif. Jika bilangan belum mencapai 1, maka program akan mencetak nilai bilangan saat ini terlebih dahulu, kemudian memanggil kembali fungsi yang sama dengan nilai bilangan dikurangi 1. Proses ini akan terus berulang hingga mencapai kondisi dasar, sehingga menghasilkan output berupa deret bilangan menurun dari *n* hingga 1.

SOAL LATIHAN

Statement rekursif

1.

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)

    fmt.Print("n  : ")
    for i := 0; i <= n; i++ {
        fmt.Print(i, " ")
    }

    fmt.Println()

    fmt.Print("Sn : ")
    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}
```

Output:

```
10
n : 0 1 2 3 4 5 6 7 8 9 10
Sn : 0 1 1 2 3 5 8 13 21 34 55
```

Deskripsi Program:

Program ini bertujuan untuk menampilkan deret Fibonacci hingga suku ke-n berdasarkan input dari pengguna. Program menggunakan fungsi rekursif untuk menghitung setiap nilai dalam deret, di mana nilai dasar (base-case) ditentukan saat n

bernilai 0 atau 1. Jika $n = 0$, maka fungsi mengembalikan 0, dan jika $n = 1$, maka mengembalikan 1. Untuk nilai n lainnya, fungsi akan memanggil dirinya sendiri dengan rumus penjumlahan dua suku sebelumnya, yaitu $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$. Di dalam fungsi utama, dilakukan perulangan dari 0 hingga n untuk mencetak seluruh deret Fibonacci secara berurutan.

2.

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    bintang(n, 1)
}

func bintang(n int, i int){
    if i > n {
        return
    }
    fmt.Println(generateBintang(i))
    bintang(n, i+1)
}

func generateBintang(jumlah int) string {
    result := ""
    for i := 0; i < jumlah; i++ {
        result += "*"
    }
    return result
}
```

Output:

```
5
*
**
***
****
*****
```

Deskripsi Program:

Program ini digunakan untuk menampilkan pola bintang berbentuk segitiga siku-siku berdasarkan input jumlah baris dari pengguna. Program memanfaatkan rekursi untuk mengatur jumlah baris yang dicetak, dimulai dari 1 hingga n. Fungsi rekursif akan memanggil dirinya sendiri dengan menambah nilai baris setiap kali pemanggilan hingga mencapai batas maksimum. Untuk mencetak jumlah bintang dalam setiap baris, digunakan fungsi tambahan yang membentuk string berisi karakter bintang sesuai jumlah yang diinginkan. Proses akan berhenti ketika jumlah baris melebihi nilai n sebagai base-case.

3.

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    faktor(n, 1)
}

func faktor(n int, i int){
    if i > n {
        return
    }
    if n % i == 0 {
        fmt.Print(i, " ")
    }
    faktor(n, i+1)
}
```

Output:

```
12
1 2 3 4 6 12
```

Deskripsi Program:

Program ini bertujuan untuk menampilkan semua faktor dari suatu bilangan bulat positif yang diinput oleh pengguna. Program menggunakan fungsi rekursif dengan parameter tambahan sebagai penghitung yang dimulai dari 1 hingga n. Pada setiap pemanggilan fungsi, program akan memeriksa apakah nilai penghitung dapat membagi

habis bilangan tersebut. Jika iya, maka nilai tersebut dicetak sebagai faktor. Fungsi akan terus memanggil dirinya sendiri dengan menaikkan nilai penghitung hingga melebihi n, yang menjadi kondisi base-case untuk menghentikan rekursi.

4.

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    pola(n)
}

func pola(n int){
    if n == 1 {
        fmt.Print("1 ")
    } else {
        fmt.Print(n, " ")
        pola(n-1)
        fmt.Print(n, " ")
    }
}
```

Output:

9

9 8 7 6 5 4 3 2 1 2 3 4 5 6 7 8 9

Deskripsi Program:

Program ini digunakan untuk menampilkan pola bilangan dari n menurun hingga 1, kemudian kembali naik hingga n. Program memanfaatkan rekursi dengan mencetak nilai n terlebih dahulu, lalu memanggil fungsi yang sama dengan nilai n-1 hingga mencapai base-case saat n bernilai 1. Setelah mencapai kondisi tersebut, proses akan kembali (backtracking) dan mencetak nilai n lagi saat fungsi selesai dipanggil. Dengan mekanisme ini, terbentuk pola bilangan yang simetris dari n ke 1 dan kembali ke n.

5.

Source Code:

```
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    ganjil(n, 1)
}

func ganjil(n int, i int){
    if i > n {
        return
    }
    if i % 2 != 0 {
        fmt.Print(i, " ")
    }
    ganjil(n, i+1)
}
```

Output:

```
20
1 3 5 7 9 11 13 15 17 19
```

Deskripsi Program:

Program ini bertujuan untuk menampilkan semua bilangan ganjil dari 1 hingga n yang diinput oleh pengguna. Program menggunakan fungsi rekursif dengan parameter tambahan sebagai penghitung yang dimulai dari 1. Pada setiap langkah, program akan memeriksa apakah bilangan tersebut ganjil dengan menggunakan operasi modulus. Jika bilangan tersebut ganjil, maka akan dicetak. Fungsi kemudian memanggil dirinya sendiri dengan menaikkan nilai penghitung hingga melebihi n sebagai kondisi base-case untuk menghentikan proses rekursif.

6.

Source Code:

```
package main
import "fmt"

func main(){
    var x, y int
    fmt.Scan(&x, &y)
    fmt.Println(pangkat(x, y))
}

func pangkat(x int, y int) int {
    if y == 0 {
        return 1
    } else {
        return x * pangkat(x, y-1)
    }
}
```

Output:

```
5 3
125
```

Deskripsi Program:

Program ini digunakan untuk menghitung hasil perpangkatan suatu bilangan x dengan pangkat y menggunakan metode rekursif. Program memiliki base-case ketika nilai y sama dengan 0, di mana hasilnya adalah 1. Jika y lebih dari 0, maka fungsi akan mengalikan nilai x dengan hasil pemanggilan fungsi yang sama dengan parameter y-1. Proses ini berlangsung terus hingga mencapai base-case. Dengan pendekatan ini, hasil perpangkatan diperoleh tanpa menggunakan library tambahan seperti math.

DAFTAR PUSTAKA

1. Munir, Rinaldi. 2011. *Algoritma dan Pemrograman dalam Bahasa Pascal dan C*. Bandung: Informatika.
2. Sedgewick, Robert & Wayne, Kevin. 2011. *Algorithms*. Addison-Wesley.
3. Kernighan, Brian W. & Ritchie, Dennis M. 1988. *The C Programming Language*. Prentice Hall.
4. Modul Praktikum Algoritma dan Pemrograman 2. (Tanpa Tahun). *Modul 5: Rekursif*.
5. Cormen, Thomas H., et al. 2009. *Introduction to Algorithms*. MIT Press.